

Wisdom & Chance TCG — Complete Mobile App AI Agent Sync Guide

Date: March 29, 2026 | Server Version: 2.5.0

Document Purpose

This document provides a comprehensive reference for an AI agent building the Wisdom & Chance TCG mobile app. It covers the current server state, all API endpoints (70+), WebSocket events, data models, authentication, game engine mechanics, economy/progression systems, and full payment integration (Stripe, PayPal, in-game currency). The mobile app should consume the existing backend as-is; no server changes are needed.

Server Connectivity

- Production URL: `https://wisdom-and-chance.replit.app`
- API base path: `/api/*` (all endpoints)
- WebSocket: `wss://wisdom-and-chance.replit.app/ws?token=<jwt>`
- Live API Docs (JSON): `GET /api/api-docs`
- Full Data Dump for Sync: `GET /api/admin/sync?code=4838`

Table of Contents

1. Project Progress Summary
2. Mobile Authentication
3. Complete API Reference (70+ endpoints)
4. WebSocket Protocol
5. Game Engine & Combat Mechanics
6. Key Data Models
7. Economy Constants & Pack Types
8. Payment System & Premium Store
9. Purchase Flows (Step-by-Step)
10. Battle Pass Premium Track
11. Payment Error Handling Reference
12. Mobile App Implementation Checklist
13. Critical Implementation Notes
14. Key Server File Reference

1. Project Progress Summary

Completed Features (Tasks 1-8)

- Task 1 - Combat Mechanic Update: Guardian & Quick Strike rework. Server-authoritative combat engine resolves all damage, healing, shields, and traits.
- Task 2 - API Docs & Rules Page: `/api/docs` endpoint + `/rules` page synchronized with 8-step combat resolution flow.
- Task 3 - Server Config & Feature Flags: `GET /api/config` returns feature flags, versioning, and season info for web and mobile.
- Task 4 - Economy & Collection System: Gold/Gems/Dust currencies, card rarity (Common/Rare/Epic/Legendary), pack opening, crafting, disenchanting, starter rewards.
- Task 5 - Mobile Sync Guide & Roadmap: Comprehensive documentation for mobile development team.
- Task 6 - Shop & Pack Opening: 7 pack types, daily deals with discounts, animated pack opening with rarity glow effects.

- Task 7 - Ranked Seasons & Battle Pass: 30-day seasons, 6 ranked tiers, 50-level battle pass, weekly challenges, season end rewards, soft rating reset.

- Task 8 - Payment Integration: PayPal + Stripe (with Google Pay) + in-game currency (gold/gems) purchases. Premium store with 11 products, premium battle pass track, purchase history, one-time purchase enforcement.

Current Server Version: 2.5.0

All 8 tasks are merged and live. The server is fully functional with all systems active.

2. Mobile Authentication

Login Flow

- Endpoint: POST /api/mobile/auth/login
- Request Body: { email: string, firstName?: string, lastName?: string, profileImage?: string, provider?: string }
- Response: { token: string, user: { id, email, firstName, lastName, profileImage } }
- Token Type: JWT (7-day expiry), signed with server SESSION_SECRET
- Auto-Provisioning: New users get: starter decks seeded, 500 gold + starter card collection (2x every Common & Rare card)

Token Refresh

- Endpoint: POST /api/mobile/auth/refresh
- Header: Authorization: Bearer <current_token>
- Response: { token: string } -- fresh 7-day token

Current User

- Endpoint: GET /api/mobile/auth/me
- Header: Authorization: Bearer <token>
- Response: Full user profile object from database

Using the Token

All /api/* endpoints accept the Authorization: Bearer <token> header. The server has a unified auth middleware that works identically for both web sessions and mobile JWT tokens. Every endpoint that works on web will work on mobile with the Bearer token.

JWT Payload: { userId: string, email: string, iat: number, exp: number }

Server maps this to req.user.claims.sub = userId for all endpoint handlers.

3. Complete API Reference

3.1 Player & Profile

- GET /api/auth/user -- Get current authenticated user
- PATCH /api/user/profile -- Update firstName, lastName, profileImage
- GET /api/player-stats -- Game statistics (XP, level, wins/losses, damage dealt)
- GET /api/player-rating -- ELO rating and current rank tier
- GET /api/users/search?q=term -- Search users by email or name

3.2 Cards & Deck Building

- GET /api/cards -- List all cards (id, name, element, power, rarity, trait, buff/debuff)
- GET /api/cards/:id -- Single card details
- GET /api/cards/element/:element -- Filter by element (fire, water, earth, air, nature)
- GET /api/commanders -- List all commander cards with abilities
- GET /api/commanders/:id -- Single commander details

- GET /api/user-decks -- List saved decks
- POST /api/user-decks -- Create deck: { name, commanderId, cardIds[] } validates 40 cards
- PATCH /api/user-decks/:id -- Update deck
- DELETE /api/user-decks/:id -- Delete deck
- POST /api/deck-suggestions -- AI deck suggestion via Gemini (commander + playstyle)
- GET /api/cards/:id/image -- Download card artwork (binary image)
- GET /api/commanders/:id/image -- Download commander artwork (binary image)

3.3 Economy & Collection

- GET /api/currencies -- Player gold, gems, dust balances
- GET /api/collection -- Player owned cards with quantities
- POST /api/collection/starter -- Grant starter collection (500g + common/rare x2)
- POST /api/packs/open -- { packType } Open pack, 5 cards with rarity-weighted pulls
- POST /api/cards/craft -- { cardId } Craft card using dust
- POST /api/cards/disenchanted -- { cardId } Disenchant card for dust

3.4 Shop (Base Gold Shop)

- GET /api/shop/catalog -- Pack catalog: Standard(100g), Premium(250g), Element(150g)
- GET /api/shop/daily-deals -- Daily deal with 15-30% discount, rotates UTC midnight
- POST /api/shop/purchase -- { packType } Buy pack with gold
- POST /api/shop/purchase-bundle -- { bundleId } Buy bundle with gems

3.5 Premium Store & Payments (NEW - Task 8)

- GET /api/payments/config -- Payment provider availability (Stripe/PayPal enabled, keys)
- GET /api/payments/products -- Full product catalog (gems, packs, bundles, battle pass)
- GET /api/payments/history -- User purchase transaction history
- POST /api/payments/purchase -- Unified purchase (routes gold/gems/stripe/paypal)
- POST /api/payments/purchase-currency -- Buy with gold or gems directly
- POST /api/payments/check-purchased -- Check one-time purchase status (single or batch)
- POST /api/payments/stripe/create-checkout -- Create Stripe checkout session
- GET /api/payments/stripe/verify-session -- Verify + fulfill Stripe payment
- GET /api/payments/stripe/success -- Alias for verify-session
- POST /api/payments/paypal/create-order -- Create PayPal order
- POST /api/payments/paypal/capture-order -- Capture PayPal payment after approval

3.6 Social & Friends

- GET /api/friends -- List friends with online status
- DELETE /api/friends -- { friendId } Remove friend
- GET /api/friend-requests -- Pending incoming/outgoing requests
- POST /api/friend-requests -- { email } Send friend request
- POST /api/friend-requests/:id/accept -- Accept request
- POST /api/friend-requests/:id/decline -- Decline request
- GET /api/friend-messages/:friendId -- Chat history with friend
- POST /api/friend-messages/:friendId -- { message } Send message

3.7 Multiplayer & Rooms

- GET /api/rooms -- List public waiting rooms
- POST /api/rooms -- { name, isPrivate?, password? } Create room
- GET /api/rooms/:id -- Room details (host, guest, spectators)
- POST /api/rooms/:id/join -- Join room
- POST /api/rooms/:id/leave -- Leave room
- POST /api/rooms/:id/ready -- { ready: bool, deckId } Set ready + select deck
- POST /api/rooms/:id/start -- Start game (host only, both ready)

- POST /api/rooms/:id/spectate -- Join as spectator
- DELETE /api/rooms/:id/spectate -- Leave spectating
- GET /api/rooms/:id/messages -- Room chat messages
- POST /api/rooms/:id/messages -- Send room chat
- GET /api/leaderboard -- Top 100 players by ELO rating

3.8 Quests & Achievements

- GET /api/achievements -- All possible achievements
- GET /api/player-achievements -- Player unlocked achievements
- POST /api/achievements/:id/claim -- Claim achievement reward (gold + BP XP)
- GET /api/daily-challenges -- Today active challenges
- GET /api/player-challenges -- Progress on daily challenges
- POST /api/player-challenges/:id/claim -- Claim daily challenge reward

3.9 Ranked Seasons & Battle Pass

- GET /api/season/current -- Active season (id, name, number, dates, daysRemaining)
- GET /api/season/rewards -- Reward table per tier (gold, packs, dust, cosmetic)
- GET /api/season/player-rank -- Player rating, tier, win/loss this season
- GET /api/season/history -- Past season records
- GET /api/battlepass -- BP progress (level, XP, rewards, claimed, premiumUnlocked, isPremium per level)
- POST /api/battlepass/claim -- { level } Claim BP level reward (enforces premium check)
- GET /api/weekly-challenges -- Active weekly challenges with progress
- POST /api/weekly-challenges/:id/claim -- Claim completed weekly challenge reward

3.10 Configuration & System

- GET /api/config -- Feature flags, server version, active season
- GET /api/health -- Server + database health check
- GET /api/api-docs -- Full API documentation as JSON

3.11 AI Chat & Image

- GET /api/conversations -- List AI chat sessions
- POST /api/conversations -- Create new conversation
- POST /api/conversations/:id/messages -- Send message, get AI response
- POST /api/generate-image -- Generate image from prompt

4. WebSocket Protocol

Connection

Connect: wss://wisdom-and-chance.replit.app/ws?token=<jwt_token>

All messages are JSON: { type: string, payload?: object }

Send "ping" periodically for keepalive. Server responds with "pong".

4.1 Client to Server Events

- join_room { roomId } -- Join a game room
- leave_room { roomId } -- Leave room
- join_game { gameId } -- Join active game session
- leave_game { gameId } -- Leave game session
- room_message { roomId, message } -- Chat to room
- game_message { gameId, message } -- Chat to game
- game_action { gameId, action: "draw"|"deploy"|"end_turn"|"use_ability"|"forfeit", data? }

- `player_ready { roomId, ready: bool, deckId? } -- Toggle ready`
- `game_start { roomId, gameId } -- Start game`
- `ping {} -- Heartbeat`

4.2 Server to Client Events

- `auth_success { userId, displayName } -- On WS connection`
- `game_state SanitizedGameState -- Full game state per player (opponent hand hidden)`
- `combat_result { gameId, ...combatData } -- Combat outcome with damage`
- `game_over { gameId, winnerId, reason } -- Game ended`
- `game_error { gameId, error } -- Action failed`
- `user_joined_room { userId, roomId }`
- `user_left_room { userId, roomId }`
- `room_message { roomId, senderId, senderName, message, timestamp }`
- `game_message { gameId, senderId, senderName, message, timestamp }`
- `player_ready { playerId, ready, deckId? }`
- `game_start { gameId } -- Game began`
- `presence_update { userId, status: "online"|"offline" }`
- `opponent_disconnected { gameId, disconnectedPlayerId, reconnectTimeout: 60 }`
- `opponent_reconnected { gameId, reconnectedPlayerId }`
- `error { message } -- General error`
- `pong {} -- Heartbeat response`

5. Game Engine & Combat Mechanics

Server-Authoritative Model

The server is the single source of truth. Clients send actions, the server validates them, updates state, and broadcasts sanitized state back. NEVER compute game logic locally for PvP. The opponent hand and face-down cards are hidden from each player.

Game Phases (per turn)

1. Draw Phase: Both players draw cards simultaneously from their deck.
2. Deployment Phase: Players select cards from hand to place face-down on battlefield.
3. Combat Phase: Cards revealed, power calculated with buffs/debuffs/traits, damage dealt.
4. Cleanup: Battlefield cleared, cards to Yard. Turn increments, back to Draw.

Combat Resolution (8 Steps)

- Step 1: Reveal all face-down cards
- Step 2: Calculate base power for each card
- Step 3: Apply buff modifiers from friendly cards
- Step 4: Apply debuff modifiers from enemy cards
- Step 5: Apply commander ability effects (shield, first_strike, heal, group buffs)
- Step 6: Calculate total power per side, determine winner
- Step 7: Apply traits: Quick Strike (pre-combat dmg), Guardian (blocks), Restoration (heals)
- Step 8: Apply base damage (power difference) to loser HP

Victory Conditions

- Reduce opponent HP to 0 (Starting HP: 40)
- Opponent forfeits
- Opponent disconnects for 60+ seconds

Commander Abilities

6. Key Data Models

6.1 Card

{ id, name, element (fire|water|earth|air|nature), power (1-10), trait (quick_strike|care_package|restoration|guardian|null), traitValue, buffElement, buffAmount, debuffElement, debuffAmount, isCommander }
Rarity derived from power: Common (1-3), Rare (4-6), Epic (7-8), Legendary (9-10).

6.2 Commander

{ id, name, element, abilities: [{ name, description, phase, costType (victory|withdrawal), costAmount, effectType, effectValue, targetElement }] }

6.3 Deck

{ id, userId, name, commanderId, cardIds: string[] }. Rules: 40 cards, max 3 copies, 1 commander. Server validates.

6.4 Currencies

{ userId, gold, gems, dust }. Starter: 500g/0gems/0dust. Match: Win=30g, Loss=10g, Draw=15g.
Currency exchange: 100 gold = \$1 USD. 100 gems = \$1 USD.

6.5 Collection

{ userId, cardId, quantity }. Starter: 2x every Common+Rare card (power 1-5).

6.6 Player Rating

{ userId, rating (ELO, starts 1000), highestRating, wins, losses, streak }
Tiers: Bronze(0+), Silver(1100+), Gold(1300+), Platinum(1500+), Diamond(1700+), Master(2000+).

6.7 Season

{ id, name, seasonNumber, startsAt, endsAt, isActive }. 30-day default. Auto-transitions.

6.8 Battle Pass

50 levels/season. XP = level*200 (cumulative). Sources: Win(+100), Loss(+30), Daily(+150), Weekly(+300), Achievement(+200).
Rewards: gold, gems, dust, packs, specific cards.
Premium track: Levels 10, 20, 25, 30, 35, 40, 45, 50 are premium (require premiumUnlocked).
Each level now has isPremium boolean. Progress object includes premiumUnlocked boolean.

6.9 Weekly Challenges

3/week/season. Types: win_games, play_element, deal_damage. Grants gold + BP XP.

6.10 PurchaseProduct (NEW)

```
{
  id: string,           // e.g. "gems_100", "premium_battle_pass"
  name: string,         // Display name
  description: string,  // Display description
  productType: string,  // "gems" | "pack_bundle" | "battle_pass" | "bundle"
  priceUsd: number,     // USD price (whole dollars)
  priceGold: number,    // Gold price (0 = not purchasable with gold)
  priceGems: number,    // Gems price (0 = not purchasable with gems)
  gemsAmount: number,   // Gems granted on purchase
  packsJson: string,    // JSON array of {type, count} for pack products
  dustAmount: number,   // Dust granted on purchase
  isOneTimePurchase: boolean, // Can only buy once per account
  isCurrencyPurchasable: boolean, // Can buy with gold/gems (false = real money only)
  isActive: boolean,    // Currently available
  badgeText: string | null, // UI badge ("Most Popular", etc.)
  sortOrder: number     // Display order
}
```

6.11 PurchaseTransaction (NEW)

```
{
  id: string,           // UUID
  userId: string,
  productId: string,    // References PurchaseProduct.id
  paymentMethod: string, // "gold" | "gems" | "stripe" | "paypal"
  paymentId: string | null, // Stripe session ID or PayPal order ID
  amountUsd: number,    // USD charged (0 for currency purchases)
  currencySpent: number, // Gold/gems spent (0 for real-money purchases)
  status: string,       // "pending" | "completed" | "failed"
  createdAt: string     // ISO timestamp
}
```

7. Economy Constants & Pack Types

Pack Types

- Standard: 100 gold / 100 gems, 5 cards. Common 60%, Rare 25%, Epic 10%, Legendary 5%
- Premium: 250 gold / 250 gems, 5 cards. Guaranteed 1+ Rare or better
- Element Packs: 150 gold / 150 gems each (Fire/Water/Earth/Air/Nature), filtered to element

Crafting/Disenchant (Dust)

- Common: 40 craft / 5 disenchant
- Rare: 100 craft / 20 disenchant
- Epic: 400 craft / 100 disenchant
- Legendary: 1600 craft / 400 disenchant

Season End Rewards (by Peak Tier)

- Bronze: 100g, 1 pack, 50 dust
- Silver: 200g, 2 packs, 100 dust, silver_card_back cosmetic
- Gold: 400g, 3 packs, 200 dust, gold_card_back cosmetic
- Platinum: 600g, 5 packs, 400 dust, platinum_card_back cosmetic
- Diamond: 1000g, 8 packs, 600 dust, diamond_card_back cosmetic
- Master: 1500g, 12 packs, 1000 dust, master_card_back cosmetic

Feature Flags (GET /api/config)

Fetch on startup, show/hide UI accordingly:

- economy_enabled -- currencies, collection, crafting

- shop_enabled -- shop catalog, purchases, daily deals
- multiplayer -- rooms, matchmaking, PvP
- ranked_seasons -- season display, ranking
- battle_pass -- BP progress, claims
- weekly_challenges -- weekly quest system
- achievements -- achievement tracking
- daily_challenges -- daily quest system

Match Rewards

- Win: 30 gold + 100 BP XP
- Loss: 10 gold + 30 BP XP
- Draw: 15 gold

8. Payment System & Premium Store

8.1 Payment Methods Overview

The server supports four payment methods:

Method	Description	Use Case
Gold	In-game currency earned from matches	Packs, bundles, battle pass
Gems	In-game currency (purchased or earned)	Packs, bundles, battle pass
Stripe	Credit/debit card + Google Pay + Apple Pay	Real-money purchases
PayPal	PayPal checkout	Real-money purchases

Key Rules:

- All USD prices are whole dollars (no cents)
- 100 gold = \$1 USD; 100 gems = \$1 USD
- Gem packs are real-money ONLY (isCurrencyPurchasable: false)
- All shop packs support BOTH gold AND gems
- Payments route to PayPal account: reagle2808@aol.com
- One-time purchases enforced with transactional SELECT FOR UPDATE lock

8.2 Payment Configuration

Before showing payment UI, fetch provider availability:

GET /api/payments/config (no auth required)

Response:

```
{
  "paypalClientId": "AaBbCc123...",
  "stripePublishableKey": "pk_live...",
  "paypalEnabled": true,
  "stripeEnabled": true
}
```

Fetch on app startup alongside feature flags. Only show enabled payment methods.

8.3 Complete Product Catalog

Gem Packs (Real Money Only)

ID	Name	USD	Gems Granted	Badge
gems_100	Starter Gems	\$1	100	--
gems_550	Popular Gems	\$4	550 (50 bonus)	Most Popular
gems_1400	Value Gems	\$8	1,400 (200 bonus)	Great Value

gems_3600	Mega Gems	\$18	3,600 (600 bonus)	--
gems_8000	Ultimate Gems	\$35	8,000 (1,500 bonus)	Best Value

All gem packs: isCurrencyPurchasable: false

Pack Bundles (Gold, Gems, or Real Money)

ID	Name	USD	Gold	Gems	Contents
premium_pack_bundle	Premium Pack Bundle	\$3	300	300	3x Premium packs
element_mega_pack	Element Mega Pack	\$6	600	600	5x Element packs (1 each)
legendary_hunt	Legendary Hunt Pack	\$8	800	800	5x Premium packs (boosted Legendary)

Battle Pass

ID	Name	USD	Gold	Gems	Effect
premium_battle_pass	Premium Battle Pass	\$5	500	500	Unlocks premium reward track

Bundles

ID	Name	USD	Gold	Gems	Contents	One-Time?
starter_bundle	Starter Bundle	\$10	1,000	1,000	1,500 gems + 5 premium packs + 500 dust	Yes
season_pass_bundle	Season Pass Bundle	\$15	1,500	1,500	Premium BP + 2,000 gems + 10 premium packs	No

8.4 Payment API Endpoints (Detailed)

GET /api/payments/products

List all active products. Response: Array of PurchaseProduct, sorted by sortOrder.

GET /api/payments/history

User purchase transaction history. Response: Array of PurchaseTransaction, sorted by createdAt desc.

POST /api/payments/check-purchased

Check one-time purchase status.

Single check: { "productId": "starter_bundle" } -> { "purchased": true }

Batch check: { "productIds": ["starter_bundle", "season_pass_bundle"] } -> { "starter_bundle": true, "season_pass_bundle": false }

POST /api/payments/purchase (Unified)

Routes to correct handler based on paymentMethod.

Request: { "productId": "gems_550", "paymentMethod": "stripe" }

For gold/gems -- immediate fulfillment:

```
{ "message": "Purchase successful", "transaction": {...}, "cards": [...], "currencies": { "gold": 450, "gems": 100,
```

For stripe -- redirect to checkout:

```
{ "action": "redirect", "url": "https://checkout.stripe.com/...", "sessionId": "cs_..." }
```

For paypal -- redirect to approval:

```
{ "action": "paypal_approve", "orderId": "ORDER_ID", "approveUrl": "https://www.paypal.com/checkouts/..." }
```

Rate limited: 10 requests per 60 seconds per user.

POST /api/payments/purchase-currency

Buy with gold or gems directly.

Request: { "productId": "premium_pack_bundle", "currencyType": "gold" }

Response:

```
{
  "message": "Purchase successful",
  "transaction": { "id": "...", "productId": "premium_pack_bundle", ... },
  "cards": [ { "cardId": "fire_001", "rarity": "Rare", "isNew": true, "cardName": "Flame Serpent", "element": "fire" },
  "currencies": { "gold": 200, "gems": 100, "dust": 0 }
}
```

POST /api/payments/stripe/create-checkout

Create Stripe Checkout session.

Request: { "productId": "gems_550" }

Response: { "sessionId": "cs_test_...", "url": "https://checkout.stripe.com/..." }

Rate limited: 5 requests per 60 seconds.

GET /api/payments/stripe/verify-session?session_id=cs_test_...

Verify completed Stripe session and fulfill purchase.

Response: { "message": "Purchase successful", "transaction": {...}, "cards": [...] }

Or: { "message": "Purchase already completed", "alreadyFulfilled": true }

GET /api/payments/stripe/success?session_id=cs_test_...

Alias for verify-session. Identical behavior.

POST /api/payments/paypal/create-order

Create PayPal order.

Request: { "productId": "gems_550" }

Response: { "orderId": "ORDER_ID_123", "approveUrl": "https://www.sandbox.paypal.com/checkout/..." }

Rate limited: 5 requests per 60 seconds.

POST /api/payments/paypal/capture-order

Capture PayPal payment after user approval.

Request: { "orderId": "ORDER_ID_123", "productId": "gems_550" }

Response: { "message": "Purchase successful", "transaction": {...}, "cards": [...] }

Server validations: status=COMPLETED, metadata match, amount >= price, idempotent.

8.5 Pack Opening Response

When a purchase includes packs, the cards array contains:

```
[
  { "cardId": "fire_001", "rarity": "Rare", "isNew": true, "cardName": "Flame Serpent", "element": "fire", "power": 10 }
]
```

Use this array to drive pack opening animation. isNew indicates if player already owned the card.

8.6 Rate Limits

- /api/payments/purchase: 10 requests per 60 seconds per user
- /api/payments/purchase-currency: 10 requests per 60 seconds per user
- /api/payments/stripe/create-checkout: 5 requests per 60 seconds per user
- /api/payments/paypal/create-order: 5 requests per 60 seconds per user
- /api/payments/paypal/capture-order: 5 requests per 60 seconds per user

9. Purchase Flows (Step-by-Step)

9.1 Gold/Gems Purchase Flow

1. Mobile: GET /api/payments/products -> display catalog
2. Mobile: GET /api/currencies -> check user balances
3. User taps "Buy with Gold" or "Buy with Gems"
4. Mobile: POST /api/payments/purchase-currency
 { productId: "premium_pack_bundle", currencyType: "gold" }
5. Server: Validates balance, deducts currency, grants items atomically
6. Response: { transaction, cards, currencies }
7. Mobile: Show pack opening animation with cards array
8. Mobile: Update local currency display from currencies object

9.2 Stripe Purchase Flow

1. Mobile: GET /api/payments/config -> confirm stripeEnabled
2. Mobile: GET /api/payments/products -> display catalog
3. User taps "Buy with Card"
4. Mobile: POST /api/payments/stripe/create-checkout
 { productId: "gems_550" }
5. Response: { sessionId, url }
6. Mobile: Open url in in-app browser (SafariVC / Chrome Custom Tab)
7. User completes payment on Stripe checkout page
8. Stripe redirects to success URL with session_id parameter
9. Mobile: Intercept redirect, extract session_id
10. Mobile: GET /api/payments/stripe/verify-session?session_id=<id>
11. Response: { transaction, cards }
12. Mobile: Show success + pack opening if applicable

Deep Link / URL Scheme:

The Stripe success_url redirects to:

https://wisdom-and-chance.replit.app/shop?payment=success&session_id={CHECKOUT_SESSION_ID}&product_id={PRODUCT_ID}

For mobile:

- Detect the redirect in the WebView
- Extract session_id from the URL query parameters
- Close the WebView
- Call /api/payments/stripe/verify-session with the session_id

9.3 PayPal Purchase Flow

1. Mobile: GET /api/payments/config -> confirm paypalEnabled
2. Mobile: GET /api/payments/products -> display catalog
3. User taps "Buy with PayPal"
4. Mobile: POST /api/payments/paypal/create-order
 { productId: "gems_550" }
5. Response: { orderId, approveUrl }
6. Mobile: Open approveUrl in in-app browser
7. User approves payment in PayPal
8. PayPal redirects back (user closes PayPal UI)
9. Mobile: POST /api/payments/paypal/capture-order
 { orderId: "ORDER_ID", productId: "gems_550" }
10. Response: { transaction, cards }
11. Mobile: Show success + pack opening if applicable

9.4 One-Time Purchase Flow (e.g., Starter Bundle)

1. Mobile: POST /api/payments/check-purchased
 { productIds: ["starter_bundle"] }
2. Response: { "starter_bundle": false }
3. If false -> show "Buy" button
4. If true -> show "Already Owned" badge, disable purchase
5. Purchase via any method above
6. Server enforces one-time with transactional SELECT FOR UPDATE lock

10. Battle Pass Premium Track

The battle pass has a premium track. Certain levels (milestones and late-game) are marked as premium. Players must purchase the Premium Battle Pass product to claim premium rewards.

10.2 Premium Levels

Premium levels: 10, 20, 25, 30, 35, 40, 45, 50

10.3 Updated Battle Pass API Response

GET /api/battlepass response:

```
{
  "season": { "id": "uuid", "name": "Season 1", "endsAt": "ISO date", "daysRemaining": 30 },
  "progress": {
    "currentXp": 1500,
    "currentLevel": 7,
    "xpIntoCurrentLevel": 100,
    "xpForNextLevel": 1600,
    "claimedLevels": [1, 2, 3],
    "premiumUnlocked": false      // NEW: true if user bought premium BP
  },
  "levels": [
    {
      "level": 1,
      "xpRequired": 200,
      "rewardType": "gold",
      "rewardAmount": 50,
      "rewardDescription": "50 Gold",
      "isPremium": false,          // NEW: true for premium-track levels
      "claimed": false,
      "unlocked": true
    }
  ]
}
```

10.4 Claiming Rewards

POST /api/battlepass/claim { "level": 10 }

Errors:

- 400 -- "Level not unlocked yet" (insufficient XP)
- 400 -- "Reward already claimed"
- 403 -- "Premium battle pass required to claim this reward"

10.5 Mobile UI for Premium Track

```
For each battle pass level:
  if level.isPremium === true:
    Show a "Premium" badge/crown icon on the level card
    if progress.premiumUnlocked === false:
      - Dim/grey out the level
      - Show lock icon instead of claim button
      - Tapping should prompt "Upgrade to Premium Battle Pass"
    if progress.premiumUnlocked === true:
      - Show level normally with premium styling (gold/amber accent)
      - Allow claiming when level.unlocked === true
  else:
    Show as normal free-track level
```

10.6 Purchasing Premium Battle Pass

Product ID: premium_battle_pass (USD: \$5, Gold: 500, Gems: 500)

After purchase, GET /api/battlepass returns premiumUnlocked: true.

HTTP Status Codes

Code	Meaning	Action
200	Success	Process response normally
400	Bad request / validation error	Show error message to user
401	Unauthorized	Re-authenticate, refresh token
403	Forbidden (premium required)	Show upgrade prompt
429	Rate limited	Show "Please wait", retry after delay
500	Server error	Show generic error, allow retry
503	Payment provider not configured	Hide that payment method

Common Error Messages

Error	Cause	Mobile Action
"Product not found"	Invalid productId	Refresh product catalog
"Insufficient gold/gems"	Not enough currency	Show "Not enough" dialog
"This product can only be purchased with real"	Gem packs	Show Stripe/PayPal only
"This product cannot be purchased with [curre"	Price is 0	Show other currency
"This one-time purchase has already been clai"	Duplicate buy	Show "Already Owned"
"Payment not completed"	Stripe unpaid	Allow retry
"Payment does not belong to this user"	Mismatch	Security error
"Payment amount mismatch"	Tampered	Security error
"Payment already fulfilled"	Duplicate call	Treat as success
"PayPal is not configured yet"	No credentials	Hide PayPal
"Stripe is not configured yet"	No credentials	Hide Stripe
"Too many requests"	Rate limited	Show cooldown
"Premium battle pass required"	Premium level	Show upgrade prompt

12. Mobile App Implementation Checklist

Phase 1: Core Foundation

- Auth screen with email login -> POST /api/mobile/auth/login
- Secure JWT storage (iOS Keychain / Android EncryptedSharedPreferences)
- Auto-refresh token before expiry (POST /api/mobile/auth/refresh)
- Fetch + cache all cards/commanders (GET /api/cards, GET /api/commanders)
- Download card artwork (GET /api/cards/:id/image), cache locally
- Fetch feature flags on startup (GET /api/config)
- Fetch payment config on startup (GET /api/payments/config)

Phase 2: Collection & Deck Building

- Collection screen: GET /api/collection merged with card data
- Deck builder: 40-card validation, max 3 copies, commander selection
- Deck CRUD: GET/POST/PATCH/DELETE /api/user-decks
- Card detail view: stats, element, rarity badge, artwork

Phase 3: Shop, Economy & Payments

3a. Payment Configuration

- Fetch GET /api/payments/config on startup
- Store stripeEnabled, paypalEnabled, stripePublishableKey, paypalClientId
- Only show enabled payment methods in UI

3b. Currency Bar & Base Shop

- Currency bar: GET /api/currencies (gold/gems/dust)
- Base shop screen: GET /api/shop/catalog
- Daily deals with countdown: GET /api/shop/daily-deals
- Crafting/disenchanted UI: POST /api/cards/craft, /disenchant

3c. Premium Store / Product Catalog

- Fetch GET /api/payments/products to get full catalog
- Group products by productType in UI:
 - "Gem Packs" for productType: "gems"
 - "Pack Bundles" for productType: "pack_bundle"
 - "Battle Pass" for productType: "battle_pass"
 - "Special Bundles" for productType: "bundle"
- Show badgeText as ribbon/badge on product cards
- Show dual pricing: gold AND gems (when isCurrencyPurchasable)
- Show USD price for all products
- Disable gold/gems buttons when isCurrencyPurchasable === false

3d. One-Time Purchase Tracking

- On shop load, call POST /api/payments/check-purchased with one-time product IDs
- Show "Already Owned" state for purchased items
- Disable buy buttons for already-purchased items

3e. In-Game Currency Purchase Flow

- "Buy with Gold" button -> POST /api/payments/purchase-currency { currencyType: "gold" }
- "Buy with Gems" button -> POST /api/payments/purchase-currency { currencyType: "gems" }
- Show confirmation dialog before purchase
- Smart default: if can't afford gold, default to gems (and vice versa)
- After purchase: update currency display from response currencies
- After purchase: trigger pack opening animation if cards array non-empty

3f. Stripe Purchase Flow

- "Pay with Card" button -> POST /api/payments/stripe/create-checkout
- Open returned url in in-app browser (SFSafariViewController / Chrome Custom Tabs)
- Intercept success redirect URL, extract session_id query parameter
- Call GET /api/payments/stripe/verify-session?session_id=<id> to fulfill
- Handle alreadyFulfilled: true as success (idempotent)
- On cancel redirect, show "Payment cancelled" message

3g. PayPal Purchase Flow

- "Pay with PayPal" button -> POST /api/payments/paypal/create-order
- Open returned approveUrl in in-app browser
- After user returns: POST /api/payments/paypal/capture-order { orderId, productId }
- Handle capture errors (user cancelled, payment failed)

3h. Purchase History

- Purchase history screen: GET /api/payments/history
- Show transaction list with: product name, payment method, amount, date
- Show "No purchases yet" empty state

3i. Pack Opening Animation

- 5 card reveals with rarity-based effects
- Use cards array from purchase response

Phase 4: Multiplayer

- WebSocket via wss://...?token=<jwt>
- Room browser (GET /api/rooms) + create room
- Pre-game lobby: ready system + deck selection
- Game board: render all phases (draw/deploy/combat/cleanup)
- Display server-sent sanitized game state (never compute locally)
- Combat animation, in-game chat, spectator mode
- Disconnect handling: 60s reconnect window

Phase 5: Social

- Friends list with online status
- Friend request send/accept/decline
- Direct messaging
- Leaderboard: GET /api/leaderboard

Phase 6: Progression

- Season info: GET /api/season/current
- Player rank with tier badge: GET /api/season/player-rank
- Season history: GET /api/season/history
- Battle pass: 50 levels, progress bar, claims (GET /api/battlepass)
- Premium track: check premiumUnlocked, show isPremium badges
- Premium-locked levels: lock icon + dim styling + upgrade prompt
- Premium-unlocked levels: gold/amber accent + claim button
- After purchasing premium BP: refresh battle pass data
- Weekly challenges with progress (GET /api/weekly-challenges)
- Daily challenges + achievements with claim flows
- Player stats: GET /api/player-stats

Phase 7: Polish

- Push notifications: friend requests, challenge completions, season transitions, purchase confirmations
- Offline mode with cached card data
- Profile editing: PATCH /api/user/profile
- App icon/splash screen, dark theme matching web app
- Loading/skeleton states for all data fetches
- Client-side purchase button debouncing (prevent rate limit hits)

13. Critical Implementation Notes

- Server-Authoritative: NEVER compute game logic on mobile for PvP. Send actions via WebSocket, render what server sends back.
- Unified Auth: Every /api/ endpoint works with Bearer token. No separate mobile endpoints needed except /api/mobile/auth/*.
- Element System: 5 elements: fire, water, earth, air, nature. Cards have buff/debuff modifiers targeting specific elements.
- Card Rarity: Derived from power: Common(1-3), Rare(4-6), Epic(7-8), Legendary(9-10). Calculated, not stored.
- Deck Rules: Exactly 40 cards, max 3 copies, 1 commander. Server validates on save.
- Starting HP: 40 HP per player. Guardian blocks damage. Restoration heals.
- ELO Rating: Starts 1000. Updated after ranked matches. Soft reset at season end toward 1000.
- Admin Email: redeagle28089@gmail.com -- only this user has admin access.
- Payment Idempotency: Stripe verify-session and PayPal capture-order are idempotent. Duplicate calls return success without double-fulfillment.
- One-Time Purchases: Server uses transactional SELECT FOR UPDATE locks. Safe against race conditions.
- Sandbox vs Production: PayPal uses sandbox (api-m.sandbox.paypal.com). Mobile does NOT need to know -- server handles all

PayPal API calls. Stripe environment determined by publishable key prefix (pk_test_ vs pk_live_).

- **WebView Payments:** Use SFSafariViewController (iOS) or Chrome Custom Tabs (Android) for Stripe/PayPal checkout. Intercept redirect URLs to extract session_id/orderId.

14. Key Server File Reference

Server-side files for reference (mobile app does NOT modify these):

- shared/schema.ts -- Core game types (Card, Commander, Deck, Game, GameState)
- shared/models/economy.ts -- Economy DB tables + constants (currencies, packs, seasons, BP, purchase products/transactions)
- shared/models/multiplayer.ts -- Social DB tables (friends, rooms, ratings, achievements)
- shared/models/auth.ts -- User, session, deck, card image tables
- server/routes.ts -- All REST API endpoints (~2900 lines)
- server/paymentRoutes.ts -- Payment endpoints (Stripe, PayPal, currency, products, history)
- server/paymentService.ts -- Product catalog, fulfillment logic, currency purchase
- server/stripeClient.ts -- Stripe client initialization via Replit connector
- server/multiplayerRoutes.ts -- Social, room, leaderboard endpoints
- server/websocket.ts -- WebSocket server: game actions, chat, presence
- server/gameEngine.ts -- Server-authoritative combat engine
- server/economyService.ts -- Helpers: grantGold, grantBattlePassXP, openPackForUser
- server/seasonService.ts -- Season transition checker, rewards distribution
- server/mobileAuth.ts -- Mobile JWT auth endpoints
- server/unifiedAuth.ts -- Unified auth middleware (session + JWT)
- server/apiDocs.ts -- API documentation generator

Quick Reference Card -- All Payment Endpoints

PAYMENT CONFIG:	GET	/api/payments/config
PRODUCT CATALOG:	GET	/api/payments/products
PURCHASE HISTORY:	GET	/api/payments/history
CHECK PURCHASED:	POST	/api/payments/check-purchased
BUY WITH CURRENCY:	POST	/api/payments/purchase-currency
UNIFIED PURCHASE:	POST	/api/payments/purchase
STRIPE CHECKOUT:	POST	/api/payments/stripe/create-checkout
STRIPE VERIFY:	GET	/api/payments/stripe/verify-session?session_id=
STRIPE SUCCESS:	GET	/api/payments/stripe/success?session_id=
PAYPAL CREATE:	POST	/api/payments/paypal/create-order
PAYPAL CAPTURE:	POST	/api/payments/paypal/capture-order
BATTLE PASS:	GET	/api/battlepass
CLAIM REWARD:	POST	/api/battlepass/claim
CURRENCIES:	GET	/api/currencies

This document was generated on March 29, 2026 for server version 2.5.0.

For the most current API reference, check GET /api/api-docs or GET /api/admin/sync?code=4838.

The mobile app should be a pure consumer of these APIs. No server modifications required.